

The Mathematics of Our LUT Mechanisms

DeepSpike Inc.

Draft, September 6, 2026

Abstract

A compact reference for the lookup-table mechanisms currently in the codebase and how they differ from one another. It follows the notation of the LUTGPT research report [1], condensing its account of **FastMHL** rather than restating it, and adds the two layers that report does not cover: **LightMHL**, whose address is non-differentiable by construction so that gradient reaches the input only through a confidence score, and **CompressionMHL**, the projection wrapper both live inside. A separate section treats the confidence score itself, since that is where the current experimental work sits. The last section states the mechanism of LookupFFN [2] and its relationship to ours; the two turn out to share their score function exactly, and to differ in which vector they take signs of.

1 Shared setup and notation

We keep the notation of the LUTGPT report [1]. A multi-head LUT layer is parameterised by an input width E , a per-head output width D_{out} , a head count H , a number of tables per head tph , and a number of anchor pairs N (NAP) per table. The layer holds $H \cdot \text{tph}$ independent tables; the per-head output is the sum of that head’s tph tables.

Each table fixes N *anchor pairs* $(a_j, b_j)_{j=1}^N$ once at module initialisation and never moves them. They are drawn from the module’s **random_seed** and stored as **register_buffers**, so they are not parameters and receive no gradient. For an input x the pair differences, or *margins*, are

$$d_j = x_{a_j} - x_{b_j}, \quad m_j := |d_j|, \quad j = 1, \dots, N. \quad (1)$$

The map $x \mapsto (d_1, \dots, d_N)$ is linear and sparse: each d_j depends only on x_{a_j} and x_{b_j} , so any gradient with respect to d returns to x as a scatter-add at the anchor positions. A weight table $W \in \mathbb{R}^{K \times D_{\text{out}}}$ with $K = 2^N$ rows is indexed by $b \in \{0, 1\}^N$, and we write $\chi_j(b) := 2b_j - 1 \in \{-1, +1\}$ for the bit parity.

The encoding is purely *rank-coded*: each bit asks only whether one coordinate exceeds another, so it is invariant to any positive rescaling of x . Both layers below share equation (1) and the table shape; they differ in what they do between the margins and the output.

2 FastMHL

FastMHL is derived in full in Section 2 of the research report [1]; we condense it here and do not restate the derivation.

Soft sign and row score. Two scalar temperatures, shared by all tables and stored in log-space, control the layer: T_{soft} (per-anchor sign sharpness) and T_{sel} (row-selection sharpness). For each

anchor pair,

$$p_j = \frac{d_j}{T_{\text{soft}} + |d_j|} = \text{sign}(d_j) \frac{|d_j|}{T_{\text{soft}} + |d_j|} \in (-1, +1), \quad S(b) = \sum_{j=1}^N p_j \chi_j(b), \quad (2)$$

with $z(b) = S(b)/T_{\text{sel}}$ and $\pi(b) = \text{softmax}_b z(b)$ the full- K distribution over rows. The argmax of S is the hard sign-pack,

$$b^* = \arg \max_b S(b), \quad b_j^* = [d_j > 0]. \quad (3)$$

Forward modes. In *hard* mode the output is the single selected row, $y^{\text{hard}} = W[b^*]$, and the temperatures play no role in the forward. In *hybrid-smooth* mode the layer takes an exact two-row softmax over b^* and its least-confident Hamming-1 neighbour $b^{\text{alt}} = b^* \oplus e_{j^*}$, $j^* = \arg \min_j |d_j|$. With $d_{\min} := |d_{j^*}|$ the score gap is $\Delta = 2|p_{j^*}| = 2d_{\min}/(T_{\text{soft}} + d_{\min})$, and

$$u = \sigma(-\Delta/T_{\text{sel}}) \in (0, \frac{1}{2}], \quad y^{\text{hyb}} = (1 - u) W[b^*] + u W[b^{\text{alt}}]. \quad (4)$$

As $d_{\min}/T_{\text{soft}} \rightarrow \infty$ or $T_{\text{sel}} \rightarrow 0^+$, $u \rightarrow 0$ and $y^{\text{hyb}} \rightarrow y^{\text{hard}}$.

What makes it differentiable through the address. This is the load-bearing point for the comparison that follows, so we state it exactly as the report does. The backward splits in two. Weight gradient flows only through the rows that actually contributed to the forward — one row in hard mode, two in hybrid-smooth. *Input and temperature gradients are computed as the analytic gradient of the full- K soft forward* $y_{\text{full}} = \sum_b \pi(b) W[b]$, ignoring the truncation the actual forward applied. So FastMHL is not differentiable through its address in any literal sense: the address is a discrete argmax. It is trained through a *surrogate* in which the row choice is replaced by the full softmax π , which is dense in all N anchor pairs and therefore delivers a signal to every coordinate of x that any anchor touches. The report is explicit that this is a deliberate mismatch: the forward is a sign-pack lookup, the backward is the gradient of a different, smooth function.

The confidence gate. Recent work added an opt-in gate (`forward_confidence`). When it is set, each gathered row is scaled by the smooth scalar s of Section 4, computed from the same margins:

$$y \mapsto s(m) \cdot y. \quad (5)$$

The gate is off by default, and the layer is bit-identical to the published one when it is.

3 LightMHL

LightMHL is not covered by the research report. What follows is derived from the source, `src/spiky/lutorch/light_multi_head_lut.py`.

Its defining property is that **the address is a non-differentiable integer and there is no straight-through estimator**. Consequently gradient reaches the input *only* through the confidence score. Where FastMHL replaces the discrete choice by a dense softmax surrogate in the backward, LightMHL does not replace it at all.

Shapes. In the per-head configuration the layer receives $z \in \mathbb{R}^{B \times H \times E}$ and holds anchors of shape $[H, \text{tph}, N]$ and tables of shape $[H \cdot \text{tph}, 2^N, D_{\text{out}}]$. Anchors are drawn per head from generator seed `random_seed + h` and **indexed head-locally**: head h 's anchors address only that head's own E coordinates. (A flattened copy carrying per-head offsets is registered separately, purely so the fused CUDA kernel can take a single $[\cdot, N]$ anchor table over a flat x ; it is the same geometry.) The tables are initialised `Uniform[-ε, +ε]`. There are **no temperatures** — the layer has no $T_{\text{soft}}, T_{\text{sel}}$, and hence 2 fewer parameters per layer than the Fast configuration it is compared against.

Forward, identical in training and evaluation. With margins $d_{h,t,j}$ from (1) computed inside head h for table t ,

$$\text{address: } c_{h,t} = \sum_{j=1}^N \mathbf{1}[\text{sd}(d_{h,t,j}) > 0] 2^{N-j} \in \{0, \dots, 2^N - 1\}, \quad (6)$$

$$\text{score: } s_{h,t} = s(|d_{h,t,\cdot}|) \in \mathbb{R}, \quad (7)$$

$$\text{output: } y_h = \sum_{t=1}^{\text{tph}} s_{h,t} W_{h,t}[c_{h,t}] \in \mathbb{R}^{D_{\text{out}}}. \quad (8)$$

Here `sd(·)` denotes `.detach()`: the sign is taken of a detached tensor, so c carries no gradient. The bits are packed *MSB-first*, matching FastMHL's index convention — the leading anchor pair contributes 2^{N-1} . The sum in (8) is evaluated with `F.embedding_bag` using the scores as `per_sample_weights`, which is algebraically the gather-scale-sum written above but never materialises the $[B, H, \text{tph}, D_{\text{out}}]$ row tensor.

The gradient. Since c is detached, $\partial y_h / \partial W$ is a scatter into the addressed rows scaled by s , and the entire input gradient factors through the score:

$$\frac{\partial \mathcal{L}}{\partial d_j} = \underbrace{\left\langle \frac{\partial \mathcal{L}}{\partial y}, W[c] \right\rangle}_{g_s, \text{ scalar}} \cdot \frac{\partial s}{\partial m_j} \cdot \text{sign}(d_j), \quad (9)$$

returned to z by scatter-add at the anchor positions. Two consequences follow directly and are the reason Section 4 matters. First, if $\partial s / \partial m_j$ is small the input receives almost nothing, regardless of how informative the routing would be. Second, the whole of $\partial \mathcal{L} / \partial d$ is rank-one in the sense that a single scalar g_s multiplies a fixed profile across the N anchors: the layer can learn *how confident* to be, but nothing in the gradient tells it that a different row would have been better.

4 The confidence forms

The score is a smooth scalar built from the margins $m_j = |d_j|$. The implementation validates against exactly three names — `"bounded"`, `"margin"`, `"bounded_norm"` — and both FastMHL and LightMHL call the same function, so an ablation between the two layers holds the score fixed. Writing $P := \prod_{j=1}^N \sigma(2m_j) \in (0, 1]$,

$$\text{bounded: } s = P, \quad (10)$$

$$\text{bounded_norm: } s = \exp\left(\frac{1}{N} \sum_j \log \sigma(2m_j)\right) = P^{1/N}, \quad (11)$$

$$\text{margin: } s = \left(\sum_j m_j\right) \cdot P. \quad (12)$$

All three are computed through $\log \sigma$ so that e^{2m} is never formed. A constant `confidence_gain` c multiplies s , and by linearity multiplies $\partial s / \partial m$ by the same c ; it separates the gate’s scale from its selectivity. `bounded_norm` is the geometric mean of the same per-anchor sigmoids: it preserves `bounded`’s ordering while removing its N -dependent attenuation, which at $N = 8$ drives `bounded` down by more than two orders of magnitude at small margins.

The derivatives, as implemented in the analytic backward, are

$$\frac{\partial s}{\partial m_j} = \begin{cases} 2 s \sigma(-2m_j) & \text{bounded,} \\ \frac{2}{N} s \sigma(-2m_j) & \text{bounded_norm,} \\ P + 2 s \sigma(-2m_j) & \text{margin.} \end{cases} \quad (13)$$

Saturation, and why it is the active question. As margins grow, $\sigma(-2m_j) \rightarrow 0$ exponentially, so the first two forms saturate and their derivative collapses: a well-separated routing decision stops producing any input gradient at all. The `margin` form carries the extra term P , which tends to 1 rather than 0, so $\partial s / \partial m_j \rightarrow 1$ and the routing path keeps a gradient floor forever. Table 1 gives the numbers; they are computed by autograd from the shipped scoring function, and cross-checked against the analytic $\partial s / \partial m$ of (13) to a relative 10^{-9} .

m	bounded		bounded_norm		margin	
	s	$\partial s / \partial m$	s	$\partial s / \partial m$	s	$\partial s / \partial m$
0.05	5.769×10^{-3}	5.481×10^{-3}	0.524979	0.062344	0.002308	0.007962
0.10	8.353×10^{-3}	7.521×10^{-3}	0.549834	0.061879	0.006683	0.014370
0.40	5.136×10^{-2}	3.185×10^{-2}	0.689974	0.053477	0.164367	0.153280
0.80	2.296×10^{-1}	7.715×10^{-2}	0.832018	0.034941	1.469750	0.723430
1.50	6.779×10^{-1}	6.430×10^{-2}	0.952574	0.011294	8.135261	1.449582
3.00	9.804×10^{-1}	4.848×10^{-3}	0.997527	0.000617	23.529345	1.096748
6.00	1.000	1.229×10^{-5}	0.999994	0.000002	47.997641	1.000541

Table 1: Score and its derivative at $N = 8$ with all anchors held at the same margin m . Between $m = 0.4$ and $m = 3.0$ `bounded_norm`’s derivative falls by $87\times$ while `margin`’s rises by $7\times$.

5 CompressionMHL: the wrapper

Neither LUT layer is applied to the residual stream directly. `CompressionMHL` projects into a narrow per-head code, runs the lookup there, and projects back. Let d_{model} be the model width and E , D_{out} the effective inner input and output widths (`eff_in`, `eff_out`). For the per-head configuration used throughout the current experiments,

$$z = \text{compress}(x) \in \mathbb{R}^{B \times H \times E}, \quad \text{compress} : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{H \cdot E}, \quad (14)$$

$$z \leftarrow \text{LN}(z) \quad (\text{optional, lut_z_norm}), \quad (15)$$

$$y = \text{LUT}(z) \in \mathbb{R}^{B \times H \times D_{\text{out}}}, \quad \text{Fast or Light}, \quad (16)$$

$$y \leftarrow y + z \quad (\text{optional, inner_residual}), \quad (17)$$

$$\text{out} = \text{decompress}(\text{reshape}(y)) \in \mathbb{R}^{B \times d_{\text{model}}}. \quad (18)$$

Details that matter.

- **The lookup is the only nonlinearity.** Both compress and decompress are plain `nn.Linear` maps and their composition is a rank- $\leq H \cdot \min(E, D_{\text{out}})$ linear map of x . Everything that is not a linear projection happens inside the table lookup.
- decompress **is zero-initialised** (in `model_build.py`), so at step 0 the FFN branch contributes exactly nothing and the block is the identity plus attention. This is the standard residual-branch zero-init, and it has a practical consequence worth recording: any diagnostic that reads gradients at initialisation reads exactly 0 through this path unless decompress is first driven to a trained scale.
- **inner_residual requires** $E = D_{\text{out}}$, and the constructor raises otherwise — the skip in (17) lives in one space. Note it adds the *normalised* z when `lut_z_norm` is on, since (15) precedes it.
- **lut_z_norm is a LayerNorm over** E , i.e. over the last axis of z , so each head’s code is normalised independently over its own coordinates with a shared learnable affine. It sits strictly between compress and the lookup. At initialisation (gain 1, bias 0) it maps $z \mapsto (z - \bar{z})/\sigma_z$; the mean cancels in $d_j = z_{a_j} - z_{b_j}$ and the denominator is positive, so it rescales every margin without moving a single address. It changes the operating point on Table 1, not the routing.

6 LookupFFN, and the parallels

LookupFFN [2] replaces the dense FFN $y = \sum_i \sigma(\langle x, W_i \rangle) V_i$ with K learnable hash tables addressed by a learnable hash. Our reference implementation is `research/lookupffn/lookup_ffn.py` on branch `research/lookupffn`.

Projection. A structured $O(d \log d)$ map replaces a dense $d \times d$ hash matrix: $R = \prod_{i=1}^4 B_i H$, where each B_i is a learnable block-diagonal matrix and H is a fixed normalised Walsh–Hadamard transform, computed by the butterfly algorithm in adds only. Hence the name BH4. It requires the transformed width to be a power of two.

Address and score. With $z = \text{BH4}(x)$ and $m_j = |z_j|$ over the first `code_length` coordinates,

$$\text{code} = \sum_j \mathbf{1}[z_j > 0] 2^j, \quad \text{score} = \frac{\sum_j m_j}{\prod_j (1 + e^{-2m_j})}, \quad y = \sum_k \text{score}_k T_k[\text{code}_k]. \quad (19)$$

Over the full hypercube codebook $\{-1, +1\}^{\text{code_length}}$ the nearest code to z is exactly $\text{sign}(z)$, so no explicit codebook is needed; this is the top-1 truncation of a softmax over codes. The address is taken hard and *identically in training and evaluation*: there is no soft/hard split, no straight-through estimator, no temperature and no auxiliary load-balancing loss. Gradient reaches R only through the score.

Parallel 1: the score is the same function as ours. Since $\sigma(2m) = 1/(1 + e^{-2m})$, the denominator in (19) is $1/P$, so

$$\frac{\sum_j m_j}{\prod_j (1 + e^{-2m_j})} = \left(\sum_j m_j \right) \prod_j \sigma(2m_j), \quad (20)$$

which is exactly **margin**, equation (12). This was verified numerically against the reference implementation at $|z|$ scales 0.01, 0.1, 1, 3, 10: maximum relative difference 1.9×10^{-15} . Their bit-packing function is likewise the same map from a sign vector to an integer; ours packs MSB-first and theirs LSB-first, which relabels cells but induces the same partition.

Parallel 2: the gradient structure is Light’s, not Fast’s. LookupFFN’s address is hard with no surrogate, so its input gradient factors entirely through its score — exactly equation (9). Structurally it is LightMHL, not FastMHL. The published mechanism therefore *inherits* the property that the routing decision itself produces no gradient; what it does not inherit is saturation, because it uses the one form in Table 1 whose derivative does not decay.

The difference: which vector is signed. The two schemes take signs of different things. LookupFFN signs *coordinates* of a structured learned projection, $\mathbf{1}[z_j > 0]$. We sign *differences of pairs of coordinates*, $\mathbf{1}[z_{a_j} > z_{b_j}]$, of a dense learned projection. The difference form is invariant to any offset shared by the coordinates, and this is not a cosmetic distinction: measured on our trained codes, one layer had every code coordinate carrying a constant sign across tokens ($|\text{mean}|/\text{sd} \approx 750$), where coordinate-signing would return a constant address and route nothing, while the difference form still reached 44–61% of its cells. Conversely the coordinate form is what makes BH4 sufficient: it needs the projection to centre its own output, which a learned structured transform can do and a fixed random anchor pairing cannot.

References

- [1] LUTGPT: a rank-mediated transformer backbone via differentiable lookup tables. [doc/lutorch/lutgpt_research_report.pdf](https://lutorch.github.io/lutgpt_research_report.pdf).
- [2] Z. Zeng, M. Davies, P. Pulijala, K. Sankaralingam, V. Singh. LookupFFN: Making Transformers Compute-lite for CPU Inference. ICML 2024. <https://arxiv.org/abs/2403.07221>.